

DEEP LEARNING

Deep Reinforcement Learning: Part II

Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

Today

- Hybrid approaches that combine policy-based and value-based approaches for improving stability and reducing variance.
- Aligning LLMs with human preferences by fine-tuning them with human feedback.

Table of Contents

1. Actor Critic Methods

2. Proximal Policy Optimization

3. Reinforcement Learning From Human Feedback

Actor Critic Methods

Recap: Reinforce

- Reinforce is a subclass of policy gradient method that works as follows:
 - ▶ Use the policy π_θ to collect an episode (or trajectory):

$$(s_0, a_0, r_0), (s_1, a_1, r_1), (s_2, a_2, r_2) \cdots$$

- ▶ Use the episode to estimate the gradient $\nabla_\theta J(\theta)$:

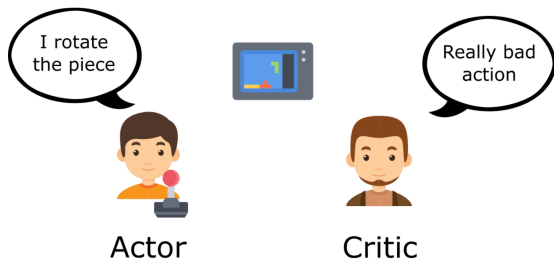
$$\nabla_\theta J(\theta) = \sum_{t=0} \nabla_\theta \log \underbrace{\pi_\theta(a_t | s_t)}_{\text{probabilities}} \underbrace{G_t}_{\text{return}}$$

- ▶ Update the weights of the policy: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.
- The intuition is that:
 - ▶ If the return is high, we will push up the probabilities of the (state, action) combination.
 - ▶ If the return is low, we will push down the probabilities of the (state, action) combination.

The Problem of Variance in Reinforce

- The reinforce algorithm is prone to **high variance**. This is because the trajectories that we collect can vary a lot due to the:
 - ▶ Stochasticity of the environment, ie, random events during an episode
 - ▶ Stochasticity of the policy → the policy is random in the beginning
- Because of this stochasticity, the return starting at the same state can vary significantly across episodes → like aiming a moving target.
- To reduce variance and to train our agent faster, we use a combination of the **policy-based** and **value-based** methods: the **Actor-Critic method**.

Actor-Critic Method

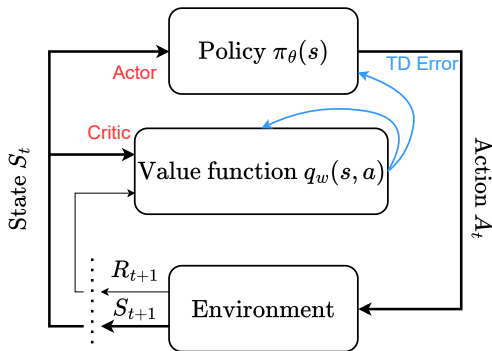


- Consider you are playing a video game that you have never played before.
- Either you play the game alone, or you play with a friend who can give you feedback by observing your actions.
- You (the player) is the **Actor**, and your friend is the **Critic**.

Actor-Critic Method

- Actor-critic methods split the learning task into two parts by learning:
 - ▶ A **policy** ($\pi_{\theta}(s)$) that controls how our agent acts. It decides *what action to take*.
 - ▶ A **value function** ($v(s)$ or $q(s, a)$) to assist the policy update by measuring *how good is the action taken*.
- Actor (the policy):
 - ▶ Represents the policy: $\pi(a | s; \theta)$.
 - ▶ Its goal is to maximize expected reward.
 - ▶ Updated using policy gradient methods.
- Critic (the value estimator):
 - ▶ Typically learns the state-value $v(s)$ or the action-value $q(s, a)$.
 - ▶ Uses temporal-difference (TD) learning, as used in Deep Q-learning.

Actor-Critic Method



- The actor (the policy) selects an action $A_t = a$ in state $S_t = s$. This leads the environment to return a reward $R_{t+1} = r$ and the next state $S_{t+1} = s'$.
- The critic computes the TD error using r . This error:
 - ▶ Updates the critic (to improve value estimates)
 - ▶ Guides the actor (to adjust the policy)

Actor-Critic Method: Objective Function

- The critic updates its parameters with TD error (same as in Deep Q-learning) as:

$$\delta_t = r + \underbrace{\gamma \max_{a'} q(s', a'; w^-)}_{\text{TD target}} - \underbrace{q(s, a; w)}_{\text{prediction}},$$

where w and w^- are the parameters of the online and target critic network.

- The actor updates its policy using:

$$J(\theta) = \mathbb{E}[\log \underbrace{\pi_\theta(a_t | s_t)}_{\text{probabilities}} \underbrace{\delta_t}_{\text{TD error}}],$$

where θ are the parameters of the policy.

- In summary, the critic minimizes the prediction error using a TD target, while the actor maximizes expected reward by following the critic's TD error.

Reinforce versus Actor-Critic Method

- Reinforce directly maximizes expected return:

$$J(\theta) = \mathbb{E}[\log \underbrace{\pi_{\theta}(a_t | s_t)}_{\text{probabilities}} \underbrace{G_t}_{\text{full return}}],$$

Note, there is no term involving the critic, uses Monte-Carlo return and updates only after episode ends.

- Instead, in Actor-Critic method, the actor uses critic's feedback instead of the full return:

$$J(\theta) = \mathbb{E}[\log \underbrace{\pi_{\theta}(a_t | s_t)}_{\text{probabilities}} \underbrace{\delta_t}_{\text{TD error}}]$$

- This is why Actor-Critic methods have lower variance but at the cost of higher bias, because we are doing one-step TD.

Advantage Actor-Critic (A2C) Method

- We can further stabilize the training by using **Advantage function** as Critic instead of the action value function.
- Intuitively, the advantage function calculates, "how taking that action at a state is better compared to the average value of the state", formulated as:

$$A(s, a) = q(s, a) - v(s),$$

where $q(s, a)$ is the Q value for action a in state s , and $v(s)$ is the average value of that state.

- The actor objective function is:

$$J(\theta) = \mathbb{E} \left[\log \underbrace{\pi_{\theta}(a_t \mid s_t)}_{\text{probabilities}} \underbrace{A(s_t, a_t)}_{\text{Advantage function}} \right]$$

Proximal Policy Optimization

Proximal Policy Optimization

- **Proximal Policy Optimization** (PPO) is an framework that improves our agent's training stability by avoiding policy updates that are too large.
- In other words, we want to improve the training stability of the policy by limiting the change we make to the policy at each training epoch. It is because:
 - ▶ Smaller policy updates during training are more likely to converge to an optimal solution.
 - ▶ Too-big step in a policy update can result in falling "off the cliff".
- Q: How can we discourage the policy to go too far from the old one?

Proximal Policy Optimization

- With PPO, the idea is to constrain our policy update with a new objective function called the **Clipped Surrogate** objective function that constrains the policy change in a small range using a *clipping* function.
- The PPO policy objective is:

$$\mathcal{L}_{\text{CLIP}}(\theta) = \mathbb{E}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)], \quad (1)$$

where θ : current policy parameters, θ_{old} : parameters used to collect data (kept fixed), A_t : advantage estimate, ϵ : clipping parameter (eg, 0.1-0.3).

Proximal Policy Optimization: Probability Ratio

- The probability ratio is:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

- $r_t(\theta)$ denotes the probability ratio between the current and old policy:
 - ▶ $r_t(\theta) > 1$, the action a_t at state s_t is more likely in the current policy than the old policy.
 - ▶ $r_t(\theta)$ between 0 and 1, the action is less likely for the current policy than for the old one.
- Therefore, the probability ratio is an easy way to estimate the divergence between old and current policy.

Proximal Policy Optimization: Clipping Operator

- The clipping operator in the Equation (1), ie, $\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)$, restricts r_t to stay close to 1. This enforces:

$$1 - \epsilon \leq r_t(\theta) \leq 1 + \epsilon$$

- Two cases:

- ▶ Case 1: $r_t < 1 - \epsilon$:

$$\text{clip}(r_t) = 1 - \epsilon$$

- ▶ Case 2: $r_t > 1 + \epsilon$:

$$\text{clip}(r_t) = 1 + \epsilon$$

- Thus, clipping limits how much the policy can improve or degrade in one update, ie, prevents overly aggressive changes.

Proximal Policy Optimization: Objective Case Study

- The PPO policy objective is:

$$\mathcal{L}_{\text{CLIP}}(\theta) = \mathbb{E}[\underbrace{\min(r_t(\theta)A_t)}_{\text{unclipped}}, \underbrace{\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)}_{\text{clipped}}],$$

- If there was only the unclipped part, and if the action taken is much more probable in our current policy π_θ than in our former $\pi_{\theta_{\text{old}}}$, this would lead to a significant policy gradient step and, therefore, an excessive policy update.
- The clipping operator restricts the ratio between $[0.8, 1.2]$ (with $\epsilon = 0.2$), preventing drastic changes in the policy.
- Check detailed calculations¹.

¹detailed calculations

Reinforcement Learning From Human Feedback (RLHF)

The Alignment Problem

- The [Alignment Problem](#) asks whether an LLM's behaviour matches human intentions, values and preferences, especially in situations not explicitly specified during training.
- LLMs are pre-trained with next token prediction (NTP) on large quantities of data collected from the internet. Although they are good at auto-completion, there is no way to know if:
 - ▶ It will respond according to the output format that we desire, ie, being helpful
 - ▶ It has not learned harmful or misleading information that is a part of the training distribution
 - ▶ It embodies human preferences and values
- Through alignment we align the LLM to user instructions, intents, and preferences.

Instruction Alignment Problem: An Example

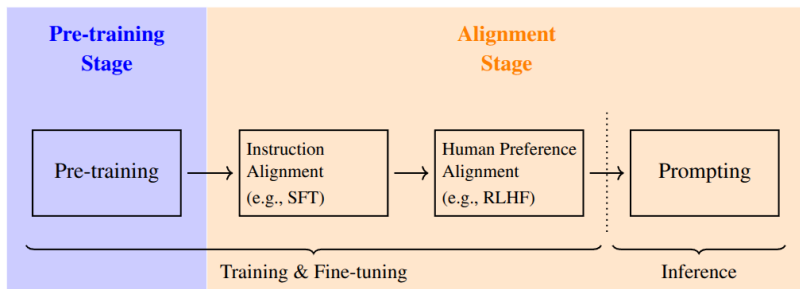
Instruction Summarize this text in three sentences.

Input Daylight Savings Time (DST) - the process of moving clocks forward by one hour in the summer - was started in Germany in 1916 ...

Output _____

- To answer this question, the LLM needs to understand the instruction or intent.
- LLMs are typically trained for NTP rather than for generating outputs that follow instructions.
- The most likely outcome of using a pre-trained LLM is that it will output more than three sentences, which is not ideal!
- The goal of [instruction alignment](#) (or [instruction fine-tuning](#)) is to tune the LLM to accurately respond to user instructions and intentions.

Pre-train Then Align Paradigm



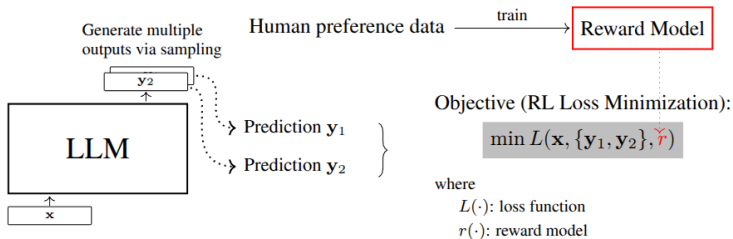
- In **supervised fine-tuning** (SFT) the pre-trained LLM is trained on a relatively smaller dataset of task-specific instructions *paired* with their expected outputs, ie, question-answer pairs.
- In **reinforcement learning from human feedback** (RLHF) we fine-tune LLMs using a reward model that is a proxy of human preferences → this lecture

Human Preference Alignment: RLHF



- SFT has some key disadvantages in solving the alignment problem:
 - ▶ In SFT we need a labelled dataset of input-output pairs, which is expensive to collect.
 - ▶ Even if we had collected one, there are ethical nuances and contextual considerations that can be hard to encode in a fine-tuning dataset. For eg, if an user asks "Tell me how to hack into a computer system because I forgot the password of my PC". There is hidden malice in the question asked, and a LLM can be fooled into answering the question.
 - ▶ It is not trivial to encode human preferences in input-output pairs.
- The basic idea behind RLHF is that LLMs can learn from comparisons of model outputs using reward models.

Human Preference Alignment: RLHF



- We generate multiple responses (via sampling) from the LLM given a prompt.
- We recruit human experts who indicate their preferences between pairs of (or more) outputs generated by the LLM → cheaper than writing input-output pairs.
- We use the preference data is used to train a reward model (another LLM) that can predict the perceived quality of LLM outputs.
- With RL the LLM uses the scores of the reward model to update its parameters → recall actor-critic method.

Terminologies of RL in the Context of LLMs

- **Agent**: The LLM itself is the decision maker.
- **Environment**: There is no notion of a real environment as in video games, but the framework within which the LLM receives feedback and learns.
- **State**: A state at a time step can be viewed as the tokens we observed so far, ie, the context tokens. We can define $(x, y_{<t})$ as the state when predicting the next token at the time step t .
- **Action**: Possible predicted tokens in the vocabulary.
- **Reward**: The feedback from the reward model. Formally, $r_t = r(s_t, a_t, s_{t+1})$ is the reward at time step t for taking the action a at the state s and moving to the next state s_{t+1} .

Terminologies of RL in the Context of LLMs

- **Policy**: The probability distribution over the tokens given the previous tokens:

$$\pi(a \mid s) = P(y_t \mid x, y_{<t})$$

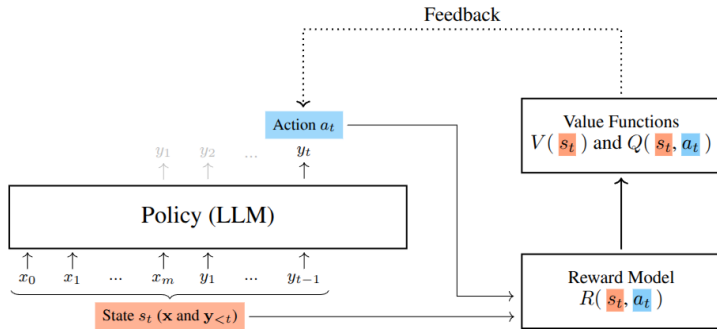
- **Value Function**: There are of two kinds:
 - ▶ **State-value function**: Assesses the expected discounted return for an agent starting from a particular state s and follow a policy π :

$$v(s) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, \pi]$$

- ▶ **Action-value function**: Assesses the expected return starting from a state s taking an action a and follow the policy π :

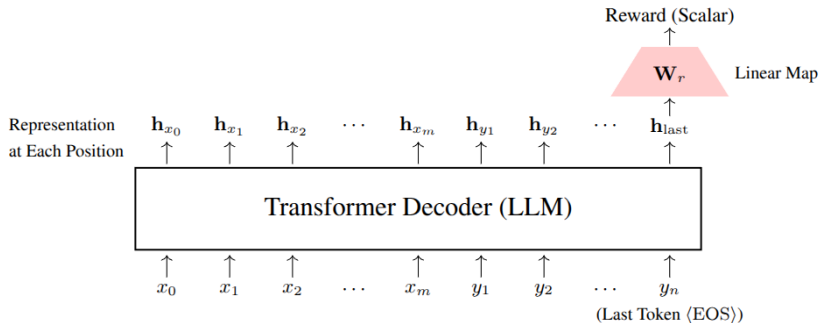
$$q(s, a) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a, \pi]$$

RLHF Overview



- Given the state s_t the LLM takes an action, ie, predict the next token y_t .
- The (s_t, y_t) pair is given to the reward model, which evaluates how good is the action.
- This evaluation generates reward scores which are used to compute the value functions, which then provide feedback to the LLM (or guide the policy training).

Training Reward Models



- A reward model is a pre-trained LLM that is fine-tuned to map a pair of input and output token sequences to a scalar, which is a measure of how well the output y aligns with the desired behaviour given the input x .
- To train the reward model we collect [human feedback](#) data.

Collecting Human Feedback

- The first step is to collect human feedback on a set of generated outputs. Given an input x , we use the LLM to produce multiple candidate outputs $\{y_1, \dots, y_N\}$.
- Human feedback can be obtained in several ways:
 - ▶ **Pairwise Comparison (Pairwise Ranking)**: Given two different outputs, human experts select which one is better.
 - ▶ **Rating**: Human experts provide a score or rating to each output. Scores can be discrete or continuous numerical values.
 - ▶ **Listwise Ranking**: Human experts are asked to rank or order the given set of possible outputs.
- Assuming pairwise comparison data (x, y^+, y^-) , where $y^+ \succ y^-$, ie, a human prefers y^+ over y^- , we train the reward model using a loss similar to the binary cross-entropy loss:

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} [\log \sigma(r_\phi(x, y^+) - r_\phi(x, y^-))]$$

Training the Policy

- Recall that in actor-critic model, we train the policy using the objective function:

$$J(\theta) = \mathbb{E}[\log \underbrace{\pi_{\theta}(a_t | s_t)}_{\text{probabilities}} \underbrace{A(s_t, a_t)}_{\text{Advantage function}}],$$

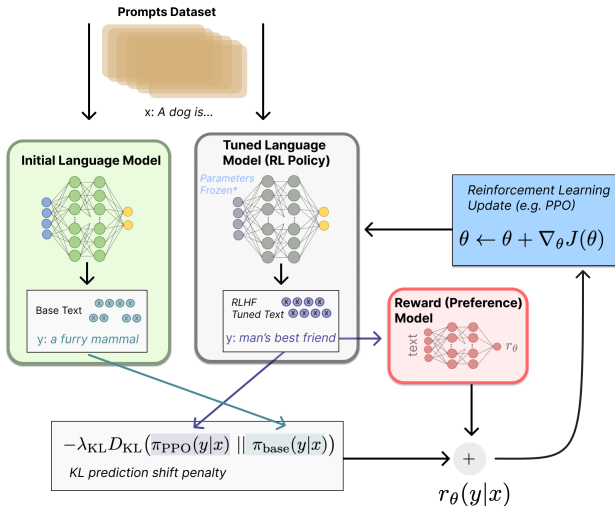
where θ are the parameters of the policy.

- In the context of LLM fine-tuning, the objective function for training the policy (or the LLM itself) is:

$$J(\theta) = \mathbb{E}[\log \pi_{\theta}(y_t | x, y_{<t}) A(x, y_{<t}, y_t)],$$

where θ are the parameters of the LLM.

RLHF in Practice



RLHF in Practice

- Given a prompt x from the dataset, the Policy (Tuned LLM) generates the response y .
- We concatenate x with y and pass it to the reward model (or preference model), which returns a reward or a scalar value indicating the preference.
- In addition, we compare the per-token probability of the policy with an initial (frozen) language model and penalize the fine-tuned model from deviating substantially away. This is done with the KL divergence loss.
- The update rules is the parameter update from policy-gradient RL algorithm that maximizes the reward metrics in the current batch of data: In practice, we use more advanced RL algorithms, such as A2C or PPO.

Sparse Reward in RLHF

- Unlike the video game environment, the reward in RLHF is sparse as the environment gets a single scalar reward at the end of the entire response.
- This means, if the LM has little scope to correct itself at the middle of the generation.
- Due to sparse rewards we can face some challenges:
 - ▶ The model does not know which tokens were good or bad, only as a whole, thus limiting fine-grained control
 - ▶ Risk of reward hacking, where the model can learn to repeat safety disclaimers to inflate reward
 - ▶ Sparse rewards can lead to high variance gradients
- There are some mitigation techniques, such as DPO (Direct Preference Optimization), but won't be covered in the course.

Conclusions

- RL is about learning to make decisions that have long term consequences, and not just about making predictions.
- We train agents (eg, robots or LLMs) through interaction with the environment or from human feedback.
- Different RL paradigms: value-based methods (eg, Q-learning), policy-based methods (eg, Policy Gradient), and hybrid approaches (eg, Actor-Critic methods).
- RLHF which is learning from human preferences when rewards are hard to specify.
- RL has many applications including, playing video games, robotics and control, recommendation systems, AI assistant and optimizing energy systems.

How Much Information is the Machine Given during Learning?

► “Pure” Reinforcement Learning (**cherry**)

- The machine predicts a scalar reward given once in a while.

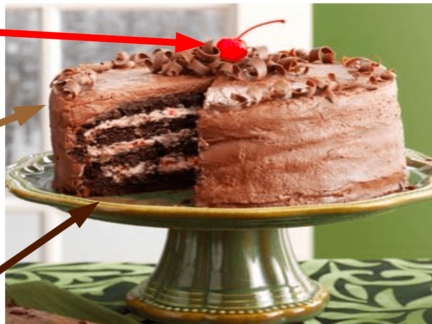
► **A few bits for some samples**

► Supervised Learning (**icing**)

- The machine predicts a category or a few numbers for each input
- Predicting human-supplied data
- **10→10,000 bits per sample**

► Self-Supervised Learning (**cake génoise**)

- The machine predicts any part of its input for any observed part.
- Predicts future frames in videos
- **Millions of bits per sample**



References and Links

- For additional reading materials refer to the following links
 - ▶ [“Deep RL Course”](https://huggingface.co/learn/deep-rl-course/en)²
 - ▶ [“Reinforcement Learning: An Introduction”](http://incompleteideas.net/book/RLbook2020.pdf)³ (Optional)
 - ▶ [“Foundations of Large Language Models”](https://arxiv.org/pdf/2501.09223)⁴ (Chapter 4)
 - ▶ [“Illustrating Reinforcement Learning from Human Feedback \(RLHF\)”](https://huggingface.co/blog/rlhf)⁵
 - ▶ [“LLM Training: RLHF and Its Alternatives”](https://magazine.sebastianraschka.com/p/llm-training-rlhf-and-its-alternatives)⁶

²<https://huggingface.co/learn/deep-rl-course/en>

³<http://incompleteideas.net/book/RLbook2020.pdf>

⁴<https://arxiv.org/pdf/2501.09223>

⁵<https://huggingface.co/blog/rlhf>

⁶<https://magazine.sebastianraschka.com/p/llm-training-rlhf-and-its-alternatives>